



Time Series Data Utilities



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>



Data Organization in tspredict

Raw Series

Original time series data ready for processing.

ts_projection()

Separation into input matrix (X) and target vector (y).

This pipeline represents the data preparation and structuring stage of `tspredict` — the phase that precedes modeling: sliding windows as context, supervised projection as interface, and temporal splitting as a valid evaluation protocol.

ts_data()

Creation of the sliding window matrix with lag columns.

ts_sample()

Temporal split between training and test sets.

What is ts_data?

`ts_data()` converts a raw time series into the tabular representation used by `tspredict`. Each row stores a forecasting context, and the lag columns (`t{k}`) define what the model can observe before predicting the next value.

Without sliding window

```
# ts_data without window (t0 only)
data <- ts_data(y)
ts_head(data)

## t0
## [1,] 0.0000000
## [2,] 0.2474040
## [3,] 0.4794255
## [4,] 0.6816388
## [5,] 0.8414710
## [6,] 0.9489846
```

With window size 10

```
# ts_data with window sw=10 (t9..t0)
data10 <- ts_data(y, 10)
ts_head(data10)

## t9    t8    ... t0
## [1,] 0.000 0.247 ... 0.778
## [2,] 0.247 0.479 ... 0.598
## [3,] 0.479 0.681 ... 0.381
## [4,] 0.681 0.841 ... 0.141
## [5,] 0.841 0.948 ... -0.108
## [6,] 0.948 0.997 ... -0.350
```

Each row slides one position forward, capturing the full historical context of 10 steps.

Projecting `ts_data` into **X** and **y**

`ts_projection()` separates the lagged attributes from the forecast target — the bridge between the time series representation and the supervised learning interface. With window `sw=5`, the first 4 columns (`t4..t1`) form **X** and the last column (`t0`) forms **y** — the value to be predicted.

Input Data – **X** (`io$input`)

```
# Input data (X)
ts_head(io$input)

## t4    t3    t2    t1
## [1,] 0.000 0.247 0.479 0.681
## [2,] 0.247 0.479 0.681 0.841
## [3,] 0.479 0.681 0.841 0.948
## [4,] 0.681 0.841 0.948 0.997
## [5,] 0.841 0.948 0.997 0.983
## [6,] 0.948 0.997 0.983 0.909
```

Output Data – **y** (`io$output`)

```
# Output data (y)
ts_head(io$output)

## t0
## [1,] 0.8414710
## [2,] 0.9489846
## [3,] 0.9974950
## [4,] 0.9839859
## [5,] 0.9092974
## [6,] 0.7780732
```

Train/Test Split with `ts_sample()`

`ts_sample()` creates train and test segments **without shuffling** the series. In forecasting, preserving chronological order is essential: the model learns from the past and is evaluated only on genuinely future observations.

Creating the Sample

```
# Sliding windows (sw=10)
sw_size <- 10
ts <- ts_data(tsd$y, sw_size)

# Sampling (train and test)
test_size <- 3
samp <- ts_sample(ts, test_size)
```

First rows of training set

```
## t9  t8  ... t0
## 0.000 0.247 ... 0.778
## 0.247 0.479 ... 0.598
## 0.479 0.681 ... 0.381
```

Test Data (`samp$test`)

```
# Test data
ts_head(samp$test)

## t9  t8  ... t0
## [1,] 0.823 0.938 ... -0.075
## [2,] 0.938 0.994 ... -0.319
## [3,] 0.994 0.989 ... -0.544
```

With `test_size=3`, the last 3 windows are reserved for testing. The training set contains all previous windows, ensuring that no future information contaminates the learning process.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

